

# Empowering VLMs for Few-Shot Multimodal Time Series Classification via Tailored Agentic Reasoning

Lin Li\*

lilin236@mail2.sysu.edu.cn  
Sun Yat-sen University  
Zhuhai, Guangdong, China

Jiawei Huang\*

huangjw255@mail2.sysu.edu.cn  
Sun Yat-sen University  
Zhuhai, Guangdong, China

Qihao Quan

quanqh@mail2.sysu.edu.cn  
Sun Yat-sen University  
Zhuhai, Guangdong, China

Dan Li†

lidan263@mail.sysu.edu.cn  
Sun Yat-sen University  
Zhuhai, Guangdong, China

Boxin Li

liboxin1@xiaomi.com  
Xiaomi Corporation  
Beijing, China

Xiao Zhang

zhangxiao16@xiaomi.com  
Xiaomi Corporation  
Beijing, China

Erli Meng

mengerli@xiaomi.com  
Xiaomi Corporation  
Beijing, China

Wenjie Feng

fengwenjie@ustc.edu.cn  
University of Science and Technology  
of China  
Hefei, Anhui, China

Jian Lou

louj5@mail.sysu.edu.cn  
Sun Yat-sen University  
Zhuhai, Guangdong, China

See-Kiong Ng

seekiong@nus.edu.sg  
National University of Singapore  
Singapore, Singapore

## Abstract

As a rich fusion of multiple types of media, time series encompass correlated and complementary representations across numeric, textual, and visual formats, and are pervasive across diverse real-world application domains. Harnessing recent advances in large vision-language models (VLMs) offers a promising direction for exploiting the rich intrinsic temporal characteristics embedded across these complementary media for few-shot time series classification. However, existing approaches rely on static context that fail to automatically evolve with the few-shot training samples, and lack tailored agentic reasoning mechanisms to maximally squeeze out discriminative knowledge from the limited labeled data.

In this paper, we propose the first **VLM agentic reasoning** framework for few-shot multimodal **Time Series Classification** (MARSTSC), which introduces a self-evolving knowledge bank as a dynamic context iteratively refined via reflective agentic reasoning. The framework comprises three collaborative roles: i) Generator conducts reliable classification via reasoning; ii) Reflector diagnoses the root causes of reasoning errors to yield discriminative insights targeting the temporal features overlooked by Generator; iii) Modifier applies verified updates to the knowledge bank to prevent context

collapse. We further introduce a test-time update strategy to enable cautious, continuous knowledge bank refinement to mitigate few-shot bias and distribution shift. Extensive experiments across 12 mainstream time series benchmark datasets demonstrate that MARSTSC delivers substantial and consistent performance gains across 5 VLM backbones, outperforming both classical and foundation model-based time series baselines under few-shot conditions, while producing interpretable rationales that ground each classification decision in human-readable feature evidence. Code is available at <https://github.com/HuangJW0821/MarSTSC>.

## CCS Concepts

• Computing methodologies → Temporal reasoning.

## Keywords

Multimodal Time Series, VLM, Agentic Reasoning

## ACM Reference Format:

Lin Li, Jiawei Huang, Qihao Quan, Dan Li, Boxin Li, Xiao Zhang, Erli Meng, Wenjie Feng, Jian Lou, and See-Kiong Ng. 2026. Empowering VLMs for Few-Shot Multimodal Time Series Classification via Tailored Agentic Reasoning. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26)*, November 10–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3767308.3836170>

## 1 Introduction

Time series classification (TSC) is a fundamental task prevalently encountered across diverse real-world domains, including healthcare monitoring [46, 75], industrial pattern recognition [1, 21], environmental sensing [20, 47], and financial analysis [2, 11], where accurately capturing temporal patterns and precisely identifying

\*Equal contribution.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MM '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2213-4/2026/11

<https://doi.org/10.1145/3767308.3836170>

contextually relevant information greatly impacts decision-making quality. While traditional TSC methods rely solely on numeric sequence representations [42, 60], the recent rise of large vision-language models (VLMs) [15, 38, 74] unlocks the opportunity to jointly process numeric, textual, and visual time series representations [48, 59]. These multimodal time series constitute correlated yet complementary sources of information across media, offering substantially greater potential for accurate and interpretable classification. Emerging work has begun to leverage VLMs for multimodal time series in tasks such as forecasting [31], retrieval [13], and causal discovery [36], exploiting their rich intrinsic knowledge acquired during extensive pretraining. Moreover, the inherent reasoning capability of VLMs supports interpretable decision making via chain-of-thought trajectories that provide grounded rationales, which can be particularly valuable in high-stakes application scenarios such as industrial fault detection [34], financial analysis [10], and medical diagnosis [44]. These promising results collectively suggest that VLMs hold significant promise for advancing multimodal time series classification.

Despite this potential, the application of VLMs to multimodal time series classification remains largely underexplored, particularly in the challenging few-shot setting where only a limited number of labeled samples are available per class [6, 30]. Performance in this setting hinges critically on two factors: how to maximally capture information from the limited labeled samples and pass it to the VLM, and how to construct the proper context to induce the VLM’s intrinsic knowledge and reasoning capabilities. Context is especially consequential for time series data, since the same time series values can carry very different meanings depending on the accompanying contextual information, which is unlike natural text, images, or audio where semantics are relatively stable across contexts. Straightforward approaches include fine-tuning VLM parameters on the limited training samples, or employing in-context learning to prompt the VLM directly [65, 73, 80]. Existing attempts have also explored summarizing statistics from the numeric modality, feeding textual and visual time series to VLMs, and designing reasoning templates tailored for time series discovering that such tailored templates often yield better performance than unguided VLM reasoning [82]. However, these approaches suffer from two fundamental limitations. First, they lack the capacity to automatically evolve the context so that the VLM can make the best use of the limited labeled samples, adapt its prediction behavior to the classification task, and mitigate few-shot bias. Second, existing methods lack a mechanism to precisely ground their decision-making process, i.e., both intermediate rationales and final predictions, to specific evidence in the multimodal time series data, limiting the ability to refine the context in a targeted and precise manner. This raises a key research question:

*How can we guide VLMs’ reasoning behavior to construct a proper and dynamic context for multimodal time series classification in the few-shot setting?*

In this paper, we answer this question by proposing MARSTSC, a **VLM agentic reasoning framework for few-shot multimodal Time Series Classification**. During the training stage, rather than updating VLM model parameters or relying on static context, MARSTSC introduces a dynamic knowledge bank that is iteratively optimized through reflective agentic reasoning over the few-shot training

samples, serving as an evolving context for classification. During the testing stage, equipped with the knowledge bank that maintains discriminative class characteristics, MARSTSC produces predictions and reasoning trajectories by grounding its decision-making process in the precise evidence locatable in the knowledge bank, while cautiously updating it to avoid few-shot bias. Both stages are non-trivial. For knowledge bank maintenance and refinement, solely relying on the VLM’s built-in reasoning capability risks context collapse, i.e., either over-rewriting existing content or incorrectly modifying the wrong knowledge entries during iterative refinement. For test-time adaptation, the few-shot setting inevitably introduces bias, and natural distribution shift during inference makes it non-trivial to enable reflective, cautious self-updating without compounding errors.

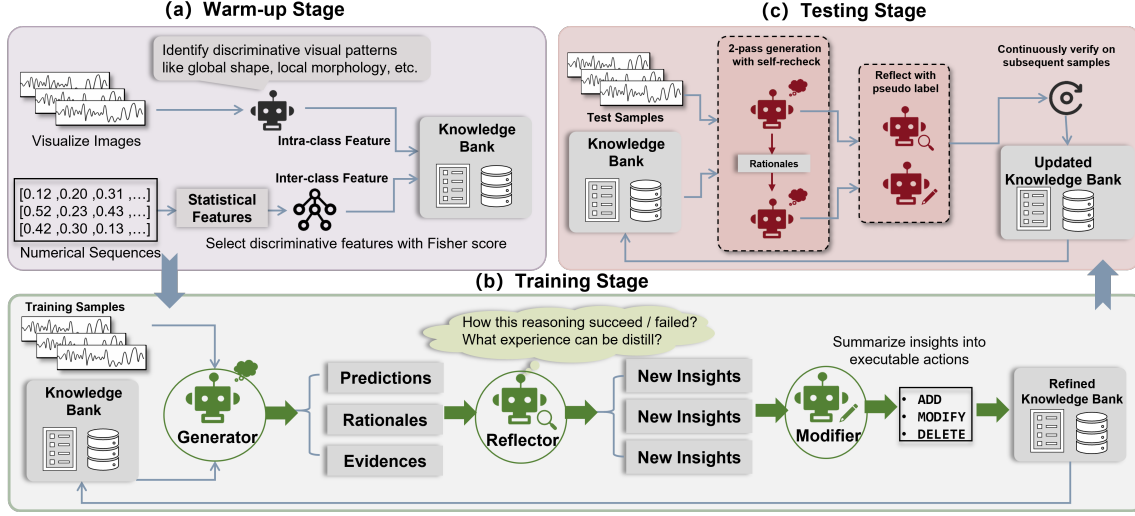
To overcome these challenges, MARSTSC devises the knowledge bank to contain structured feature bullet points and enable atomic, incremental updates with restricted actions (ADD, MODIFY, DELETE) applied on a per-bullet basis, thereby increasing the precision with which individual feature entries can be located and updated. MARSTSC proposes the agentic reasoning framework comprising three collaborative roles: the **Generator**, which performs classification by grounding its predictions in established knowledge; the **Reflector**, which diagnoses classification outcomes to extract discriminative temporal patterns and error-avoidance strategies from the reasoning trajectory; and the **Modifier**, which addresses few-shot bias and context collapse by applying verified incremental updates to the knowledge bank. Extensive experiments on 12 benchmark datasets across 7 domains corroborate the superiority of MARSTSC over both classical TSC methods, VLM/LLM-based methods, and time series foundation models in the few-shot setting. It also achieves substantial performance gains across 5 different base models, demonstrating strong transferability. Moreover, MARSTSC achieves favorable performance even when compared against methods trained under full-shot scenarios with at most 22 times more training samples.

In summary, our key contributions are as follows.

- We study the under-explored task of multimodal time series classification in the few-shot setting, proposing a tailored agentic reasoning framework to elicit the full potential of VLMs across complementary numeric, textual, and visual TS representations.
- We propose MARSTSC, a framework of time-series-tailored agentic reasoning for few-shot multimodal time series classification. It consists of three key stages: initializing task-related knowledge from visual patterns and statistical features, iteratively extracting reasoning insights and inference strategies through agentic refinement, and cautiously evolving the knowledge bank during inference via a test-time update mechanism.
- We conduct extensive experiments demonstrating that MARSTSC delivers substantial and consistent performance gains across 5 VLM backbones, outperforming both classical and foundation model-based time series baselines under few-shot conditions.

## 2 Related Work

**Time Series Classification.** Time series classification (TSC), as a foundational data analytics task, is drawing extensive research interest across numerous domains. Traditionally, TSC methods were



**Figure 1: Overview of our MARSTSC framework. It comprises three key stages: Warm-up Stage, Training Stage and Testing Stage.**

dominated by machine learning techniques trained with numerical TS data. Approaches like K-NN [17], decision tree [55], and SVM [16] showed satisfactory results via an in-domain learning paradigm. As the demand for higher performance and robustness grew, ensemble methods like random forest [9] and HIVE-COTE [42] achieved state-of-the-art performance by aggregating diverse feature representations and heterogeneous classifiers.

The paradigm subsequently shifted with the rise of deep learning, which bypassed manual feature engineering in favor of automatic representation learning. Foundational architectures such as RNN [43] and Transformer [63] model sequential dependencies through recurrent state transitions and attention mechanisms, respectively, which naturally led to more sophisticated designs, including TimesNet [67], MultiRocket [60], Autoformer [68], and PatchTST [49].

Recent progress in LLMs has fueled increasing interest in adopting large models for time series tasks [25, 27, 32, 39, 84]. Early attempts include adapting pretrained LLMs by fine-tuning on abundant datasets using textualized or patched representations of raw time series. For instance, Time-LLM [33] reprograms time series into textualized prompts, LLM4TS [12] aligns patched sequences with LLMs via two-stage fine-tuning. However, these efforts mostly focus on improving task performance via modality alignment between textual descriptions and numerical time series [14, 22, 35, 37, 66], while neglecting the reasoning capabilities of large models and the agent paradigm.

On the other hand, time series foundation models (TSFM) pre-trained with large-scale cross-domain numerical time series datasets emerge as a family of promising generalized time series analysis methods [3, 24, 40, 51, 70]. Although the aforementioned foundation models show satisfactory classification results, most of them cannot handle multimodal data and lack the reasoning ability to explicitly explain the thinking process.

Time-series data are also commonly represented through textual and visual modalities other than numerical [58, 69]. Driven by the success of works like LLaVA [38] and Qwen2.5-VL [5], researchers have shown increasing interest in leveraging Vision Language Mod-

els (VLMs) for time series analysis, unlocking a nascent yet promising frontier for addressing the aforementioned limitations of existing time series approaches [80]. Preliminary attempts suggest that VLMs could capture important temporal patterns such as trends, periodicity, and spikes during their extensive pre-training on massive imaged time series from various domains [18, 48, 85]. Furthermore, compared with LLM-based time series analysis relying on discrete textual descriptions [61], VLMs are claimed to be able to leverage imaged time series data in compact and continuous representations, which can retain more temporal information and be less restricted by the context length [18].

**Agents and Self-Improvement Techniques for Reasoning Tasks.** The evolution of Large Language Models (LLMs) has catalyzed the development of autonomous agents capable of complex reasoning, tool manipulation, and sequential decision-making. Pioneering frameworks such as ReAct [76] interleave reasoning and acting, allowing models to dynamically adjust their execution plans based on environmental feedback, and TRT [86] proposed an iterative self-improvement framework through self-generated verification signals. To maintain context over extended tasks, incorporating structured memory has become essential for agentic frameworks. For instance, architectures like A-mem [72] and ACE [77] utilized learned memory for code and math tasks.

Recently, these agentic paradigms have been extended into the domain of time series reasoning [50, 57, 71, 81]. For example, TimeXL [31] integrates a prototype-based time series encoder with three collaborating LLMs to deliver more accurate predictions and interpretable explanations; MERIT [83] utilizes three LLM agents to collaboratively generate positive views for time series data; and TSci [79] performs data analysis and forecasting by integrating specialized agents. However, these paradigms lack exploration in the reasoning ability of agentic VLMs, especially tailored for the interpretable TSC task that requires a detailed understanding of hidden time series patterns.

### 3 Problem Formulation

Time-series classification aims to learn a mapping from time-series data to class labels. Formally, let  $D = \{(x_i, y_i)\}_{i=1}^N$  denote a time series dataset with  $N$  samples, where  $x_i \in \mathbb{R}^{m \times w}$  is a multivariate time-series sample with  $m$  variables and  $w$  time steps, and  $y_i \in \{1, 2, \dots, c\}$  denotes the corresponding label among  $C$  classes. The dataset is partitioned into training, validation, and test splits, denoted by  $D_{\text{train}}, D_{\text{val}}, D_{\text{test}}$ , respectively. In particular, this paper focuses on the few-shot setting where the sample size of training and validation sets is very small per class (e.g., 3 samples), representing a realistic yet more challenging scenario as considered in previous work [64]. Unlike full-shot scenarios where models can be trained from scratch or sufficiently fine-tuned with ample labeled samples, few-shot classification demands maximally exploiting the very limited training samples, posing a fundamental challenge for capturing and refining class-discriminative knowledge from scarce examples.

During the training stage, instead of optimizing model parameters on  $D_{\text{train}}$ , our agentic multimodal time series classification framework  $\mathcal{F}$  introduces an optimizable knowledge bank  $\mathcal{K}$  that serves as dynamic contextual knowledge, updated through an iterative agentic reasoning framework. Through iterative refinement by the proposed agentic reasoning framework,  $\mathcal{K}$  is gradually enriched with task-related experience and insights during training, and continuously updated during testing.

During the testing stage, the MARSTSC framework  $\mathcal{F}$  classifies a given test sample  $x$  as:  $\hat{y} = \mathcal{F}(x, \mathcal{K}, C)$ , where  $\mathcal{K}$  denotes the refined knowledge bank, and  $C$  represents auxiliary contextual knowledge, including the dataset background and sample-level statistical characteristics. Together,  $\mathcal{K}$  and  $C$  provide the agentic context for the classification decision-making process.

## 4 Methodology

### 4.1 Overview of Our Proposed MARSTSC

We present an overview of the proposed MARSTSC for multimodal few-shot time series classification, organized into three stages: Warm-up, Training, and Testing. The overall framework of MARSTSC is illustrated in Figure 1.

**Warm-up Stage.** MARSTSC sets up the knowledge bank by extracting prior knowledge from few-shot training samples, including core statistical features (e.g., mean value, min/max value, frequency) from numerical time series and visual features (e.g., trend, periodicity, spike) from line plots. This warm-up step equips  $\mathcal{F}$  with essential domain-specific priors, allowing the agents to anchor their general pre-trained knowledge to the specific temporal patterns of the task. However, this time series knowledge remains static and suboptimal, lacking the discriminative refinement necessary for accurate classification. While previous methods have largely relied on such static contexts alone, our MARSTSC treats them only as a stepping stone for the subsequent iterative refinement during the training and testing stages, as introduced below.

**Training Stage.** Our MARSTSC optimizes the knowledge bank on the few-shot training and validation samples to gradually refine the task contexts in a dynamic manner through tailored iterative reasoning, rather than training or fine-tuning model parameters, which are known to be less effective in the few-shot setting. MARSTSC

organizes the agentic reasoning framework into three roles, i.e., Generator, Reflector, and Modifier, which cooperate to refine the knowledge bank according to the trial-and-error feedback obtained from decision-making on training and validation samples.

**Testing Stage.** MARSTSC allows agents to continuously refine the knowledge bank at test-time, so as to further increase the dynamism of the agent’s task-related knowledge, thereby mitigating distribution shift during inference. Specifically, we introduce a robust Test-Time Update mechanism designed to selectively refine the knowledge bank. By leveraging self-recheck through two-pass generation, pseudo-label-based refinement and gated knowledge management, our method is able to cautiously yet effectively adapt to previously unseen temporal patterns and extract test-time task-related reasoning strategies in an unsupervised manner.

### 4.2 Warmup by Atomic Knowledge Bank Design

**Knowledge Bank Design.** The key design choice of MARSTSC lies in introducing a dynamic knowledge bank that maximally squeezes contextual information out from the very few training and validation samples and optimizes the knowledge rather than updating model parameters to support subsequent classification. The structure of the knowledge bank is pivotal, as solely relying on the VLM’s built-in reasoning capability risks over-updating the current knowledge by completely rewriting its existing content or incorrectly updating the wrong knowledge entries. Therefore, we structure the knowledge bank as feature bullet points to enable atomic updates, thereby increasing the precision with which individual feature entries can be located. Each bullet is maintained by a unique ID and a tag counter (useful, neutral, useless). Specifically, Figure 2 provides an illustration of the knowledge bank design, which contains the following sections: (1) Intra-class features; (2) Inter-class features; (3) Mistakes to avoid; (4) Distinguishing strategies or insights; (5) Dataset background and domain knowledge clues.

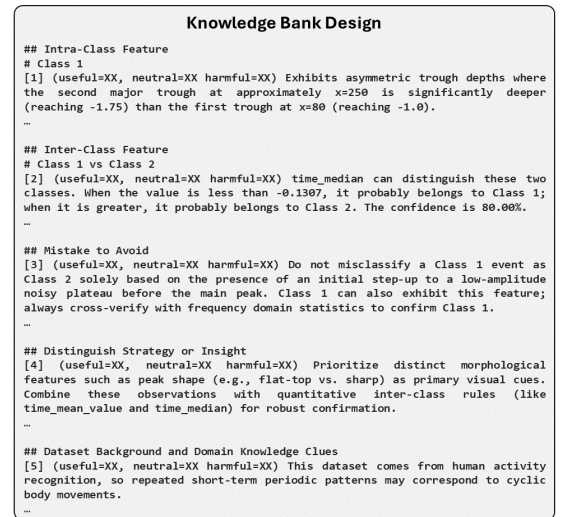


Figure 2: Illustration of knowledge bank design.

**Intra-class Feature Initialization.** We construct intra-class feature descriptors to capture prototypical characteristics of samples for each class, covering both the numeric and imaged time series modalities and organizing their complementary features jointly.

Specifically, we leverage the cross-modal alignment capabilities of VLMs to analyze the visual representations of the imaged time series modality and extract intra-class semantic descriptors, which are then combined with statistical and domain knowledge extracted from the numeric time series modality.

**Inter-class Feature Initialization.** Inter-class feature descriptors capture the most salient distinguishing characteristics between pairs of classes. The aim is to equip the knowledge bank with explicit discriminative boundaries that guide the agent in differentiating visually or semantically similar classes during classification. However, extracting such boundaries is non-trivial: while VLMs excel at capturing semantic and visual patterns, they lack the quantitative precision required for determining fine-grained classification boundaries, necessitating a complementary statistical analysis beyond semantic initialization alone. We compute a normalized Fisher Score to evaluate pairwise class separability, establishing explicit decision thresholds for the most discriminative descriptors between each pair of classes. After filtering out redundant metrics, only the most critical statistical descriptors are retained and incorporated into the inter-class feature entries of the knowledge bank.

### 4.3 Training by Iterative Reflective Reasoning

To dynamically refine the knowledge bank, thereby implicitly improving the precision of locating the decision boundaries between classes, we introduce an iterative reflective reasoning framework comprising three collaborative roles: a Generator, a Reflector, and a Modifier. Rather than updating model parameters, these three roles engage in a cooperative trial-and-error process over the few-shot training and validation samples. In detail, the Generator produces classification decisions and rationales that the Reflector diagnoses, and the Reflector’s extracted lessons are then structured and committed to the knowledge bank by the Modifier. This cooperative loop allows the framework to backtrack from suboptimal feature bullets, avoid compounding errors across reasoning rounds, and iteratively converge toward a more accurate and generalized knowledge bank through accumulated trial-and-error experience.

**Generator.** The Generator serves as the entry point of each reflective reasoning round, producing a classification decision explicitly grounded in the current state of the knowledge bank. Given a query time-series sample  $x_q$  and the current knowledge bank  $\mathcal{K}_t$ , the Generator is prompted to refer to  $\mathcal{K}_t$  for decision-making evidence and cites the specific feature bullet points  $\mathcal{T}$  from  $\mathcal{K}_t$  that directly support its classification prediction. This grounded reasoning produces a predicted class label  $\hat{y}$  alongside a chain-of-thought reasoning trajectory  $\mathcal{R}$  as the rationale that makes the knowledge bank usage explicit and traceable:

$$(\hat{y}, \mathcal{R}, \mathcal{T}) = \text{Generator}(x_q, \mathcal{K}_t, C). \quad (1)$$

**Reflector.** The Reflector serves as the evaluation component of each reasoning round, diagnosing the Generator’s decision by comparing the predicted class  $\hat{y}$  against the ground truth  $y_{gt}$  and identifying the precise causes of any reasoning errors. Upon evaluating the prediction, the Reflector tags the cited feature bullet points as either *useful*, *neutral*, or *useless* based on their contribution to the correct or incorrect decision. When a misclassification occurs

( $\hat{y} \neq y_{gt}$ ), the Reflector retrieves representative hard-positive samples  $x^+$  from the ground-truth class  $y_{gt}$  and hard-negative samples  $x^-$  from the incorrectly predicted class  $\hat{y}$ . By contrasting the differences among  $x_q$ ,  $x^+$ , and  $x^-$ , it identifies the root causes of the original reasoning error and yields concrete and discriminative insights  $\mathcal{I}_{new}$  that isolate the exact temporal features the Generator overlooked or underestimated. These contrastive samples serve as explicit comparative anchors that guide the Reflector in pinpointing fine-grained discriminative boundaries overlooked by the Generator.

**Modifier.** We find that naive self-improvement that directly prompts the model to refine the knowledge bank leads to overwriting the entire content. As the knowledge bank grows, monolithic rewriting inevitably causes context degradation, where information is suddenly erased or compressed, hampering the convergence of the iterative refinement process and yielding degraded or irrelevant feature descriptions. To ensure atomic and precise refinement, the Modifier is restricted to performing updates bullet by bullet. It reads the Reflector’s new insights  $\mathcal{I}_{new}$ , reviews the existing knowledge bank, and outputs structured, granular actions  $action \in \{\text{ADD}, \text{MODIFY}, \text{DELETE}\}$ . Instead of regenerating the entire repository, it produces a compact set of incremental updates that are applied directly to the knowledge bank in a structured and automated manner. This strategy allows the knowledge bank to incrementally accumulate new insights:

$$\mathcal{K}_{t+1} = \mathcal{K}_t \oplus \text{Modifier}(\mathcal{I}_{new}, \mathcal{K}_t, C). \quad (2)$$

**Discarding.** Real-world time series data is notoriously susceptible to transient anomalies, sensor artifacts, and atypical outliers. Spurious rules pollute the knowledge bank, causing a noise-induced bias that severely degrades generalization on subsequent samples. To safeguard the knowledge bank against such pollution, MARS-TSC incorporates a discarding mechanism driven by the Reflector’s tagging history. This is crucial because models may generate unsupported details in their rationales [28, 29]. To mitigate this, we maintain a dynamic counter for each bullet  $k_i \in \mathcal{K}$ . If the number of useless tags exceeds a threshold  $\tau_{drop}$ , or if the ratio of useful to useless tags indicates that the bullet is ambiguous, the Modifier forcefully purges or refines it. This strategy reduces spurious noise and ensures that only statistically recurrently supported temporal patterns are permanently consolidated into the knowledge bank.

### 4.4 Testing with Evolving Knowledge Bank

The knowledge extracted from few-shot samples is highly dependent on the quality of those samples, and may therefore lack sufficient generalization and transferability. This limitation becomes even more pronounced in time series classification, where distribution shifts are common and further expose the shortcomings of a static knowledge bank. To address this, we propose continuously evolving the knowledge bank at test-time to dynamically capture distributional patterns absent from the few-shot training samples. However, the test stage is inherently unsupervised, and naively treating model predictions as reliable supervision risks knowledge corruption that undermines downstream classification performance. To this end, we propose a Test-Time Update mechanism that enables cautious and reliable knowledge bank evolution during inference.

**Two-pass Generation.** To reduce reasoning bias and mitigate hallucinations, we propose a two-pass generation mechanism. Specifically, the Generator first makes a standard inference for the current test sample, producing the first-pass predicted label  $\hat{y}_1$  and corresponding reasoning  $\mathcal{R}_1$ :

$$(\hat{y}_1, \mathcal{R}_1, \mathcal{T}_1) = \text{Generator}(x, \mathcal{K}, C). \quad (3)$$

In the second pass,  $(\hat{y}_1, \mathcal{R}_1)$  is integrated into the Generator’s context and will be treated as a suspicious reference. The generator is then prompted to re-examine whether the prior reasoning was overconfident or overlooked important evidence, and to produce a new prediction:

$$(\hat{y}_2, \mathcal{R}_2, \mathcal{T}_2) = \text{Generator}(x, \mathcal{K}, C, \hat{y}_1, \mathcal{R}_1). \quad (4)$$

The second-pass prediction is taken as the final answer. This mechanism helps better exploit the self-correction ability of large models [7, 54, 62], and improves reasoning accuracy through self-recheck.

**Refine on Pseudo Label.** Another purpose of our two-pass generation design is to provide a pseudo-supervisory signal for test-time refinement. Specifically, we treat second-pass prediction as a pseudo-label and perform reflection under the hypothesis that the current test sample belongs to class  $\hat{y}_2$ . We first examine the consistency between the two passes. If the two predictions disagree, we regard this as an indication that  $\hat{y}_1$ -related knowledge is unreliable or insufficient. In this case, the test sample is treated as a negative instance and  $\hat{y}_1$ -related knowledge is sharpened accordingly. In contrast, if the two predictions are consistent, the test sample is treated as a positive instance for class  $\hat{y}_2$  and the associated knowledge is further refined.

The refinement process is carried out by the Reflector and Modifier using the same prompts as in the training stage. The only difference is that the Modifier is not allowed to perform the DELETE operation, in order to accommodate the update design introduced in the following section.

**Deferred Knowledge Update.** When the two-pass generation mechanism produces the same incorrect prediction in both passes, the system may still associate the test sample with an incorrect label, and the risk of knowledge contamination remains. To address this issue, we propose a deferred knowledge update mechanism, which continuously validates newly acquired knowledge during the test stage.

Specifically, we partition the knowledge memory into three levels according to confidence: prototype knowledge, reference knowledge, and candidate knowledge.

- *Prototype knowledge* refers to the knowledge acquired during warm-up and training stages. Since it is generated with labeled samples, it is treated as reliable ground-truth and serves as the knowledge bank foundation. Prototype knowledge is frozen during testing and the Modifier is not allowed to operate on it, thereby preventing contamination of the core knowledge.
- *Reference knowledge* refers to test-time knowledge that has been repeatedly observed in subsequent samples. It is provided to the Generator as a suspicious but informative reference.
- *Candidate knowledge* refers to newly extracted knowledge at test-time. It is temporarily stored in a buffer and does not participate in classification decisions.

To support knowledge evolution, we maintain a counter for each candidate and reference item. After the modifier completes its operation, we check whether the candidate knowledge associated with  $\hat{y}_2$  is observed in the current sample. Its counter is increased by 1 if it is observed, otherwise decreased by 1. When the score exceeds threshold  $\tau_{\text{promote}}$ , the candidate is promoted to reference knowledge; when it falls below threshold  $\tau_{\text{remove}}$ , it is then removed from the knowledge memory. For reference knowledge, we further adjust its counter according to the Reflector’s *useful/useless* judgment. If the score drops below  $\tau_{\text{promote}}$ , it is demoted back to candidate knowledge. Through this deferred update design, the system continuously filters and calibrates test-time knowledge, ensuring that only patterns and insights repeatedly supported by subsequent observations are incorporated into decision-making, thereby further improving the quality of the knowledge bank.

## 5 Experiments

### 5.1 Experimental Settings

**Dataset.** We conduct experiments on 12 datasets collected from the UCR/UEA archive [4, 19], covering diverse scenarios with varying numbers of variables and classes. For each dataset, we follow the default train–test split, and further construct few-shot train and validation sets by randomly sampling from the original training split. Specifically, we set  $k = 3$  by default, meaning that 3 examples are sampled for each class at most, in both  $D_{\text{train}}$  and  $D_{\text{val}}$ .

**Baselines.** The proposed method is compared against several representative TSC baselines, spanning into three categories: *machine learning-based methods* including DrCIF, HIVE-COTE 2.0 [42], *deep learning-based methods* including TimesNet [67], Autoformer [68], PatchTST [49], and *large model-based methods* including GPT4TS [84], MOMENT [24].

In accordance with the few-shot training manner of our proposed method, all baselines are evaluated under similar few-shot settings in the main results. Meanwhile, since most of the baselines are not designed as few-shot TSC methods, for a fair and comprehensive comparison, we also compare the proposed method (few-shot) with baselines under the full-shot setting, where they are trained or fine-tuned on the original full training split.

**Implementation Details.** Line plot is adopted as visual representation, as it explicitly presents the shape-based temporal characteristics. We use Gemini-3.1-pro [23] as the base VLM of MARS-TSC. Results with other VLMs are shown in the appendix. For test-time update, test samples are processed sequentially in a randomly permuted order, and knowledge bank is reset before each run.

**Metric.** We use classification accuracy as the evaluation metric. All results are averaged over three independent runs with different random seeds.

**Other Details.** Additional details, ablations, and analysis are provided in Appendix B and C.

**Research Questions.** To comprehensively evaluate the effectiveness of the proposed framework, we conduct experiments to answer the following four Research Questions (RQs).

**RQ1:** Can the proposed agentic TS reasoning framework outperform traditional machine learning and deep learning methods, and existing large model-based methods under a few-shot setting?

**Table 1: Classification accuracy under few-shot setting. Best results are in Bold, and second-best results are Underlined.**

| Dataset             | MARSTSC      | DrCIF        | HIVE-COTE 2.0   | TimesNet     | Autoformer | PatchTST | GPT4TS       | MOMENT       |
|---------------------|--------------|--------------|-----------------|--------------|------------|----------|--------------|--------------|
| ArrowHead           | <b>0.766</b> | 0.554        | <u>0.669</u>    | 0.606        | 0.451      | 0.303    | 0.411        | 0.606        |
| DodgerLoopGame      | <b>0.884</b> | 0.739        | <u>0.746</u>    | 0.681        | 0.580      | 0.522    | 0.529        | 0.710        |
| ECG200              | <b>0.790</b> | 0.660        | <u>0.770</u>    | 0.660        | 0.690      | 0.760    | 0.760        | 0.750        |
| GesturePebbleZ1     | <b>0.785</b> | 0.262        | <u>0.209</u>    | <u>0.302</u> | 0.157      | 0.209    | 0.168        | 0.198        |
| Lightning7          | <b>0.822</b> | 0.658        | 0.671           | 0.616        | 0.603      | 0.315    | 0.452        | <u>0.726</u> |
| UMD                 | <b>0.993</b> | 0.799        | 0.625           | 0.549        | 0.514      | 0.507    | 0.583        | <u>0.757</u> |
| Worms               | <b>0.481</b> | <u>0.260</u> | 0.442           | 0.182        | 0.208      | 0.286    | 0.260        | <u>0.442</u> |
| DiatomSizeReduction | <u>0.971</u> | 0.964        | <b>0.980</b>    | 0.804        | 0.843      | 0.284    | 0.882        | <u>0.768</u> |
| DistalPhalanxTW     | <u>0.576</u> | 0.568        | 0.554           | 0.468        | 0.511      | 0.144    | <b>0.612</b> | 0.511        |
| Fish                | <u>0.771</u> | 0.743        | <b>0.849</b>    | 0.509        | 0.509      | 0.183    | 0.286        | 0.709        |
| RacketSports        | <u>0.526</u> | 0.513        | <b>0.579</b>    | 0.401        | 0.263      | 0.250    | 0.467        | 0.520        |
| SelfRegulationSCP1  | <u>0.697</u> | 0.659        | <b>0.791</b>    | 0.677        | 0.536      | 0.529    | 0.642        | 0.689        |
| <b>1st Count</b>    | <b>7</b>     | <b>0</b>     | <u><b>4</b></u> | <b>0</b>     | <b>0</b>   | <b>0</b> | <b>1</b>     | <b>0</b>     |

**Table 2: Accuracy comparison with direct inference.**

| Dataset             | Ours         | base+text+num | base+text+plot |
|---------------------|--------------|---------------|----------------|
| DodgerLoopGame      | <b>0.884</b> | 0.607         | 0.623          |
| DiatomSizeReduction | <b>0.971</b> | 0.255         | 0.291          |
| DistalPhalanxTW     | <b>0.576</b> | 0.180         | 0.173          |
| Fish                | <b>0.771</b> | 0.143         | 0.137          |
| GesturePebbleZ1     | <b>0.785</b> | 0.164         | 0.146          |

**Table 3: Ablation on test-time update.**

| Dataset             | Full pipeline | w/o update | w/o 2-pass |
|---------------------|---------------|------------|------------|
| DodgerLoopGame      | <b>0.884</b>  | 0.812      | 0.659      |
| DiatomSizeReduction | <b>0.971</b>  | 0.791      | 0.782      |
| DistalPhalanxTW     | <b>0.576</b>  | 0.561      | 0.482      |
| Fish                | <b>0.771</b>  | 0.731      | 0.686      |
| GesturePebbleZ1     | <b>0.785</b>  | 0.767      | 0.773      |

**RQ2:** Can the performance of the proposed agentic TS reasoning framework under fixed few-shot setting compete with that of baselines under increasing  $k$ -shot settings (until full-shot settings)?

**RQ3:** Can the proposed agentic TS reasoning framework with reflection and test-time update yield performance gains compared to direct LLM/VLM inference?

**RQ4:** Can the agentic rechecking and updating pipeline during the testing stage indeed improve the quality of the knowledge bank and enhance the overall performance of MARSTSC?

## 5.2 Main Results

**5.2.1 Comparison under Few-shot Setting (RQ1).** The proposed method is compared with baselines under the same few-shot setting ( $k=3$ , 3 for training and another 3 for validation). As shown in Table 1, our method consistently achieves the best or second-best performance across all benchmarks. These results highlight the key advantage of our proposed method that it can achieve strong few-shot classification performance by leveraging multimodal information and agentic reasoning rather than relying on end-to-end parameter learning with large-scale samples.

**5.2.2 Comparison with Increasing  $K$ -shot Baselines (RQ2).** In this comparison, our proposed method remains fixed in the 3-shot setting, whereas the baselines gradually increase the  $k$  values until full-shot. The results, as shown in Figure 3, indicate that our method remains competitive with, and in most cases even outperforms, baselines trained with substantially more data. This suggests that, with

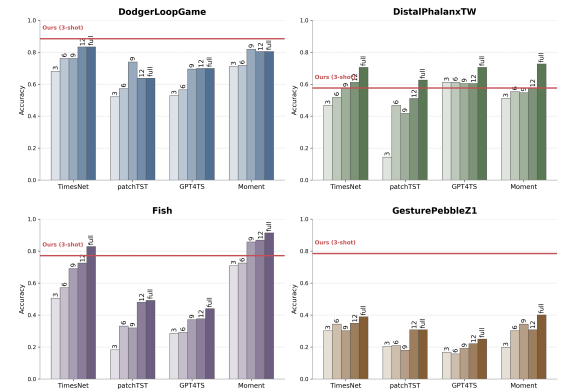
a proper reasoning procedure, few-shot samples can be exploited much more effectively, thereby unlocking stronger performance.

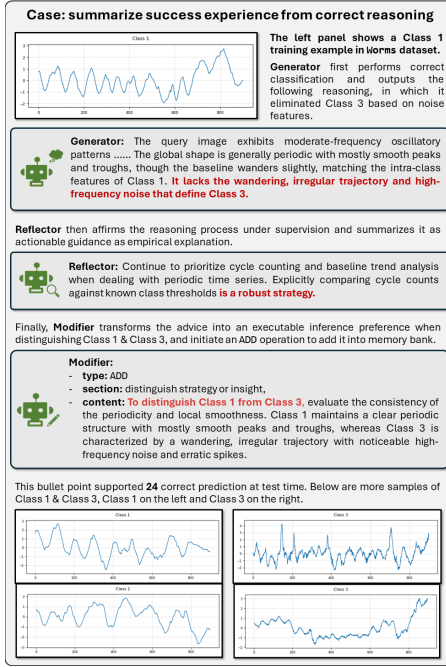
**5.2.3 Effects of TS-tailored Agentic Reasoning (RQ3).** In this section, we compare the proposed MARSTSC with a direct inference setting, where the base model (Gemini-3.1-pro, marked as “base” in the table) is prompted to conduct TS classification with merely dataset descriptions (marked as “text” in the table) and numerical time-series (marked as “num” in the table) or visualized time-series plots (marked as “plot” in the table), without any agentic reflections. This experiment aims to measure how much improvement can be brought by the proposed time-series tailored agentic TS reasoning framework.

Results in Table 2 show that directly applying the VLM base model with plain textual descriptions and TS plots yields limited performance and consistently underperforms our method, which is equipped with tailored agentic TS reasoning. This suggests that generic visual-language reasoning capabilities alone are insufficient to reliably solve time-series classification, especially in the few-shot setting. The clear advantage of our method indicates that the real performance gain does not come merely from using a stronger foundation model, but from introducing a more suitable task-adaptation reasoning paradigm for time-series classification.

## 5.3 Ablation Studies on Test Update (RQ4)

To answer RQ4, we conduct ablation studies to show the effectiveness of the test update pipeline.

**Figure 3: Comparison with Increasing  $k$ -shot Baselines**



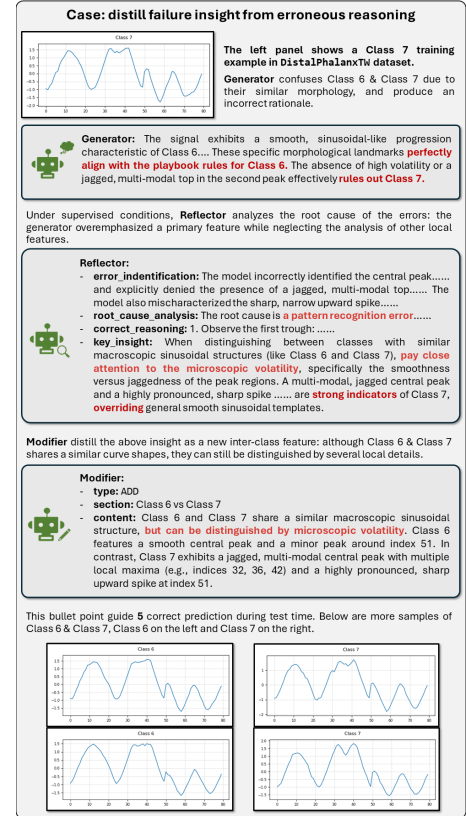
**Figure 4: A case about train refinement on success reasoning. Through refinement, the agent summarizes successful experiences into reusable and executable guidelines.**

As shown in Table 3, the ablation variant *w/o update* retains the two-pass generation for self-recheck in the testing stage but disables refine and update on knowledge bank. Its inferior performance shows that the feature descriptions derived from few-shot samples alone cannot fully cover the diversity of test examples, while test-time update helps capture additional patterns beyond the limited training data.

The ablation variant *w/o 2-pass* removes the self-recheck step and directly uses the first-pass reasoning for refinement and update, resulting in a more substantial performance drop. This suggests that the self-recheck process enables the model to correct unreliable initial reasoning before updating the feature bank. Without this step, erroneous knowledge from the mistake reasoning process is more likely to be incorporated, leading to knowledge contamination and degraded classification performance. More ablations about the key reasoning steps are in the Appendix C.6.

#### 5.4 Case Studies of Rechecking and Test-Time Update (RQ4)

This section illustrates the effectiveness of the designed rechecking and updating processes through detailed case studies. Figure 4 shows train-time refinement from a correct prediction output, where the reflector summarizes successful experience from the reasoning process and the modifier further distills it into an actionable distinguishing strategy. Figure 5 shows train-time refinement from an incorrect prediction output, where the reflector inspects the reasoning and identifies the root cause of the error upon an incorrect prediction, and then derives a corrected reasoning process and summarizes the resulting failure insight. These two cases demonstrate



**Figure 5: A case about train refinement on erroneous reasoning. Through refinement, the agent learns more accurate task knowledge.**

that the reflection process at training time could explicitly summarize successful experiences and identify error causes to improve the quality of the maintained knowledge bank.

We also present a case of a successful test-time self-recheck via two-pass generation, where the model revisits its previous reasoning, corrects the initial mistake, and ultimately arrives at the correct answer. This case suggests that the test-time rechecking and updating reasoning manner can effectively reduce the risk of error-induced knowledge contamination while further expanding the coverage and adaptability of the knowledge bank. Details can be found in Appendix C.8.

## 6 Conclusion

In this paper, we have proposed MARSTSC, the first VLM agentic reasoning framework for few-shot multimodal time series classification. Unlike existing approaches that rely on static context and suboptimally exploit limited labeled samples, MARSTSC introduces a self-evolving knowledge bank iteratively refined through three collaborative roles, i.e., a Generator, a Reflector, and a Modifier, alongside a Test-Time Update strategy that mitigates few-shot bias during inference. Extensive experiments demonstrate superior few-shot classification accuracy over both classical and VLM-based baselines, with interpretable rationales grounded in human-interpretable feature evidence.

## Acknowledgments

The authors thank the anonymous reviewers and area chairs for their constructive comments and efforts in improving this paper.

We also gratefully acknowledge the National Supercomputer Center in Guangzhou for the computational resources and support provided.

## References

- [1] Harika Abburi, Tanya Chaudhary, Sardar Haider Waseem Ilyas, Lakshmi Manne, Deepak Mittal, Don Williams, Derek Snaidauf, Edward Bowen, and Balaji Veeramani. 2023. A Closer Look at Bearing Fault Classification Approaches. In *Annual Conference of the PHM Society*, Vol. 15.
- [2] Yihao Ang, Yifan Bao, Lei Jiang, Jiajie Tao, Anthony KH Tung, Lukasz Szpurch, and Hao Ni. 2025. Structured agentic workflows for financial time-series modelling with llms and reflective feedback. In *Proceedings of the 6th ACM International Conference on AI in Finance*, 924–932.
- [3] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. 2024. Chronos: Learning the Language of Time Series. *Transactions on Machine Learning Research* 2024 (2024).
- [4] Anthony Bagnall, Hoang Anh Dau, Jason Lines, Michael Flynn, James Large, Aaron Bostrom, Paul Southam, and Eamonn Keogh. 2018. The UEA multivariate time series classification archive, 2018. *arXiv preprint arXiv:1811.00075* (2018).
- [5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *arXiv:2502.13923 [cs.CV]* <https://arxiv.org/abs/2502.13923>
- [6] Jesus Barreda, Ashley Gomez, Ruben Puga, Kaixiong Zhou, and Li Zhang. 2024. Cosco: a sharpness-aware training framework for few-shot multivariate time series classification. In *Proceedings of the 33rd ACM international conference on information and knowledge management*, 3622–3626.
- [7] Shelly Bensal, Umar Jamil, Christopher Bryant, Melisa Russak, Kiran Kamble, Dmytro Mozolevskiy, Muayad Ali, and Waseem AlShikh. 2025. Reflect, retry, reward: Self-improving llms via reinforcement learning. *arXiv preprint arXiv:2505.24726* (2025).
- [8] Donald J Berndt and James Clifford. 1994. Using dynamic time warping to find patterns in time series. In *Proceedings of the 3rd international conference on knowledge discovery and data mining*, 359–370.
- [9] Leo Breiman. 2001. Random forests. *Machine learning* 45, 1 (2001), 5–32.
- [10] Sukru Selim Calik, Andac Akyuz, Zeynep Hilal Kilimci, and Kerem Colak. 2025. Explainable-AI powered stock price prediction using time series transformers: A Case Study on BIST100. *arXiv preprint arXiv:2506.06345* (2025).
- [11] Ngai Hang Chan. 2004. *Time series: applications to finance*. John Wiley & Sons.
- [12] Ching Chang, Wei-Yao Wang, Wen-Chih Peng, and Tien-Fu Chen. 2025. Llm4ts: Aligning pre-trained llms as data-efficient time-series forecasters. *ACM Transactions on Intelligent Systems and Technology* 16, 3 (2025), 1–20.
- [13] Jialin Chen, Ziyu Zhao, Gaukhar Nurbek, Aosong Feng, Ali Maatouk, Leandro Tassiulas, Yifeng Gao, and Rex Ying. 2025. TRACE: Grounding Time Series in Context for Multimodal Embedding and Retrieval. *Advances in Neural Information Processing Systems* (2025).
- [14] Peng Chen, Yihang Wang, Yang Shu, Yunyao Cheng, Kai Zhao, Zhongwen Rao, Lujia Pan, Bin Yang, and Chenjuan Guo. 2025. Cc-time: Cross-model and cross-modality time series forecasting. *arXiv preprint arXiv:2508.12235* (2025).
- [15] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. 2024. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 24185–24198.
- [16] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine learning* 20, 3 (1995), 273–297.
- [17] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.
- [18] Mayank Daswani, Mathias MJ Bellaiche, Marc Wilson, Desislav Ivanov, Mikhail Papkov, Eva Schnider, Jing Tang, Kay Lamerigts, Gabriela Botea, Michael A Sanchez, et al. 2024. Plots unlock time-series understanding in multimodal models. *arXiv preprint arXiv:2410.02637* (2024).
- [19] Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. 2019. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica* 6, 6 (2019), 1293–1305.
- [20] Shengdong Du, Tianrui Li, Yan Yang, and Shi-Jinn Horng. 2019. Deep air quality forecasting using hybrid deep learning framework. *IEEE Transactions on Knowledge and Data Engineering* 33, 6 (2019), 2412–2424.
- [21] Mojtaba A Farahani, MR McCormick, Ramy Harik, and Thorsten Wuest. 2025. Time-series classification in smart manufacturing systems: An experimental evaluation of state-of-the-art machine learning algorithms. *Robotics and Computer-Integrated Manufacturing* 91 (2025), 102839.
- [22] Yunfeng Ge, Ming Jin, Yiji Zhao, Hongyan Li, Bo Du, Chang Xu, and Shirui Pan. 2025. EventTSF: Event-Aware Non-Stationary Time Series Forecasting. *arXiv preprint arXiv:2508.13434* (2025).
- [23] Google DeepMind. 2026. *Gemini 3.1 Pro Model Card*. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>
- [24] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. MOMENT: A Family of Open Time-series Foundation Models. In *International Conference on Machine Learning*. PMLR, 16115–16152.
- [25] Tong Guan, Zijie Meng, Dianqi Li, Shiyu Wang, Chao-Han Huck Yang, Qingsong Wen, ZuoZhu Liu, Sabato Marco Siniscalchi, Ming Jin, and Shirui Pan. 2026. TimeOmni-1: Incentivizing Complex Reasoning with Time Series in Large Language Models. In *The Fourteenth International Conference on Learning Representations*.
- [26] Tong Guan, Sheng Pan, Johan Barthelemy, Zhao Li, Yujun Cai, Cesare Alippi, Ming Jin, and Shirui Pan. 2026. TimeOmni-VL: Unified Models for Time Series Understanding and Generation. In *Forty-third International Conference on Machine Learning*.
- [27] Shiyuan Hu, Jianxin Jin, Yang Shu, Peng Chen, Bin Yang, and Chenjuan Guo. 2026. Towards Multimodal Time Series Anomaly Detection with Semantic Alignment and Condensed Interaction. In *The Fourteenth International Conference on Learning Representations*.
- [28] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [29] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM computing surveys* 55, 12 (2023), 1–38.
- [30] Yushan Jiang, Kanghui Ning, Zijie Pan, Xuyang Shen, Jingchao Ni, Wenchao Yu, Anderson Schneider, Haifeng Chen, Yuriy Nevmyvaka, and Dongjin Song. 2025. Multi-modal time series analysis: A tutorial and survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 6043–6053.
- [31] Yushan Jiang, Wenchao Yu, Geon Lee, Dongjin Song, Kijung Shin, Wei Cheng, Yanchi Liu, and Haifeng Chen. 2026. Timexl: Explainable multi-modal time series prediction with llm-in-the-loop. *Advances in Neural Information Processing Systems* 38 (2026), 135431–135467.
- [32] Ming Jin, Yaxuan Kong, Yuxuan Liang, Chaoli Zhang, Siqiao Xue, Xue Wang, James Zhang, Yi Wang, Haifeng Chen, Xiao-Li Li, Vincent Tseng, Yu Zheng, Lei Chen, Hui Xiong, Shirui Pan, and Qingsong Wen. 2026. Large Models for Time Series and Spatio-Temporal Data: A Survey and Outlook. *ACM Comput. Surv.* 58, 14, Article 368 (July 2026), 38 pages. doi:10.1145/3821637
- [33] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024. Time-LLM: Time series forecasting by reprogramming large language models. In *International Conference on Learning Representations (ICLR)*.
- [34] Mohammad Ali Labbaf Khaniki, Alireza Golkari, Houman Nouri, and Mohammad Manthouri. 2024. Enhanced fault detection and cause identification using integrated attention mechanism. *arXiv preprint arXiv:2408.00033* (2024).
- [35] Zhi Lei, Chenxi Liu, Hao Miao, Wanghui Qiu, Bin Yang, and Chenjuan Guo. 2026. MM-ISTS: Cooperating Irregularly Sampled Time Series Forecasting with Multimodal Vision-Text LLMs. *arXiv preprint arXiv:2603.05997* (2026).
- [36] Peiwen Li, Xin Wang, Zeyang Zhang, Yuan Meng, Fang Shen, Yue Li, Jialong Wang, Yang Li, and Wenwu Zhu. 2024. Realtcd: Temporal causal discovery from interventional data with large language model. In *Proceedings of the 33rd ACM international conference on information and knowledge management*, 4669–4677.
- [37] Chenxi Liu, Qianxiong Xu, Hao Miao, Sun Yang, Lingzheng Zhang, Cheng Long, Ziyue Li, and Rui Zhao. 2025. Timexma: Towards llm-empowered multivariate time series forecasting via cross-modality alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, 18780–18788.
- [38] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.
- [39] Jun Liu, Chaoyun Zhang, Jiaxu Qian, Minghua Ma, Si Qin, Chetan Bansal, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2025. Large language models can deliver accurate and interpretable time series anomaly detection. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 4623–4634.
- [40] Xu Liu, Juncheng Liu, Gerald Woo, Taha Aksu, Yuxuan Liang, Roger Zimmermann, Chenghao Liu, Junnan Li, Silvio Savarese, Caiming Xiong, et al. 2025. Moirai-MoE: Empowering Time Series Foundation Models with Sparse Mixture

- of Experts. In *International Conference on Machine Learning*. PMLR, 38940–38962.
- [41] Matthew Middlehurst, Ali Ismail-Fawaz, Antoine Guillaume, Christopher Holder, David Guijo-Rubio, Guzal Bulatova, Leonidas Tsaprounis, Lukasz Mentel, Martin Walter, Patrick Schäfer, et al. 2024. aeon: a Python toolkit for learning from time series. *Journal of Machine Learning Research* 25, 289 (2024), 1–10.
  - [42] Matthew Middlehurst, James Large, Michael Flynn, Jason Lines, Aaron Bostrom, and Anthony Bagnall. 2021. HIVE-COTE 2.0: a new meta ensemble for time series classification. *Machine Learning* 110, 11 (2021), 3211–3243.
  - [43] Navid Mohammadi Fomani, Lynn Miller, Chang Wei Tan, Geoffrey I Webb, Germain Forestier, and Mahsa Salehi. 2024. Deep learning for time series classification and extrinsic regression: A current survey. *Comput. Surveys* 56, 9 (2024), 1–45.
  - [44] Mukaffi Bin Moin, Fatema Tuj Johora Faria, Swarnajit Saha, Busra Kamal Rafa, and Mohammad Shafiq Alam. 2024. Exploring explainable ai techniques for improved interpretability in lung and colon cancer classification. In *International Conference on Computing and Communication Networks*. Springer, 1–11.
  - [45] Moonshot AI. 2026. *Kimi K2.5 Quickstart*. <https://platform.kimi.ai/docs/guide/kimi-k2-5-quickstart>
  - [46] Mohammad Amin Morid, Olivia R Liu Sheng, and Joseph Dunbar. 2023. Time series prediction using deep learning methods in healthcare. *ACM Transactions on Management Information Systems* 14, 1 (2023), 1–29.
  - [47] Ozan Baris Mulayim, Pengrui Quan, Liying Han, Xiaomin Ouyang, Dezhi Hong, Mario Berges, and Mani Srivastava. 2026. Can time-series foundation models perform building energy management tasks? *Data-Centric Engineering* 7 (2026), e9.
  - [48] Jingchao Ni, Ziming Zhao, ChengAo Shen, Hanghang Tong, Dongjin Song, Wei Cheng, Dongsheng Luo, and Haifeng Chen. 2025. Harnessing vision models for time series analysis: a survey. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 10612–10620.
  - [49] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*.
  - [50] Kanghui Ning, Zijie Pan, Yushan Jiang, Anderson Schneider, Yuriy Nevmyvaka, and Dongjin Song. 2025. Towards Interpretable and Trustworthy Time Series Reasoning: A BlueSky Vision. *arXiv preprint arXiv:2510.16980* (2025).
  - [51] Kanghui Ning, Zijie Pan, Yu Liu, Yushan Jiang, James Zhang, Kashif Rasul, Anderson Schneider, Lintao Ma, Yuriy Nevmyvaka, and Dongjin Song. 2025. Ts-rag: Retrieval-augmented generation based time series foundation models are stronger zero-shot forecaster. *Advances in Neural Information Processing Systems* (2025).
  - [52] OpenAI. 2026. *GPT-5 Model*. <https://developers.openai.com/api/docs/models/gpt-5>
  - [53] OpenAI. 2026. *GPT-5.4 mini Model*. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>
  - [54] Jing-Cheng Pang, Pengyuan Wang, Kaiyuan Li, Xiong-Hui Chen, Jiacheng Xu, Zongzhang Zhang, and Yang Yu. 2023. Language model self-improvement by reinforcement learning contemplation. *arXiv preprint arXiv:2305.14483* (2023).
  - [55] J. Ross Quinlan. 1986. Induction of decision trees. *Machine learning* 1, 1 (1986), 81–106.
  - [56] Qwen Team. 2026. *Qwen3.5-397B-A17B*. <https://huggingface.co/Qwen/Qwen3.5-397B-A17B>
  - [57] Weilin Ruan and Yuxuan Liang. 2026. Visual Reasoning over Time Series via Multi-Agent System. *arXiv preprint arXiv:2602.03026* (2026).
  - [58] ChengAo Shen, Wenchao Yu, Ziming Zhao, Dongjin Song, Wei Cheng, Haifeng Chen, and Jingchao Ni. 2025. Multi-modal view enhanced large vision models for long-term time series forecasting. *Advances in Neural Information Processing Systems* (2025).
  - [59] Zhong Siru, Ruan Weilin, Ming Jin, Li Huan, Wen Qingsong, and Liang Yuxuan. 2025. Time-VLM: Exploring Multimodal Vision-Language Models for Augmented Time Series Forecasting. In *Forty-Second International Conference on Machine Learning (ICML 2025)*. Proceedings of Machine Learning Research.
  - [60] Chang Wei Tan, Angus Dempster, Christoph Bergmeir, and Geoffrey I Webb. 2022. MultiRocket: multiple pooling operators and transformations for fast and effective time series classification. *Data Mining and Knowledge Discovery* 36, 5 (2022), 1623–1646.
  - [61] Mingtian Tan, Mike Merrill, Vinayak Gupta, Tim Althoff, and Tom Hartvigsen. 2024. Are language models actually useful for time series forecasting? *Advances in Neural Information Processing Systems* 37 (2024), 60162–60191.
  - [62] Guiyao Tie, Zenghui Yuan, Zeli Zhao, Chaoran Hu, Tianhe Gu, Ruihang Zhang, Sizhe Zhang, Junran Wu, Xiaoyue Tu, Ming Jin, Qingsong Wen, Lixing Chen, Pan Zhou, and Lichao Sun. 2025. CorrectBench: A Benchmark of Self-Correction in LLMs. In *Proceedings of the NeurIPS 2025 Datasets and Benchmarks Track*.
  - [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
  - [64] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Daan Wierstra, et al. 2016. Matching networks for one shot learning. *Advances in neural information processing systems* 29 (2016).
  - [65] Jiahao Wang, Mingyue Cheng, Qingyang Mao, Yitong Zhou, Daoyu Wang, Qi Liu, Feiyang Xu, and Xin Li. 2025. Tabletime: Reformulating time series classification as training-free table understanding with large language models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 3009–3019.
  - [66] Siyuan Wang, Peng Chen, Yihang Wang, Wanghui Qiu, Chenjuan Guo, Bin Yang, and Yang Shu. 2026. Unlocking the Value of Text: Event-Driven Reasoning and Multi-Level Alignment for Time Series Forecasting. In *The Fourteenth International Conference on Learning Representations*.
  - [67] Haixu Wu, Tenge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *International Conference on Learning Representations*.
  - [68] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* 34 (2021), 22419–22430.
  - [69] Shunyu Wu, Dan Li, Wenjie Feng, Haozheng Ye, Jian Lou, and See-Kiong Ng. 2026. Rating Quality of Diverse Time Series Data by Meta-learning from LLM Judgment. In *The Fourteenth International Conference on Learning Representations*.
  - [70] Xingjian Wu, Jianxin Jin, Wanghui Qiu, Peng Chen, Yang Shu, Bin Yang, and Chenjuan Guo. 2026. Aurora: Towards Universal Generative Multimodal Time Series Forecasting. In *ICLR*.
  - [71] Xingjian Wu, Junkai Lu, Zhengyu Li, Xiangfei Qiu, Jilin Hu, Chenjuan Guo, Christian S Jensen, and Bin Yang. 2026. TimeART: Towards Agentic Time Series Reasoning via Tool-Augmentation. *arXiv preprint arXiv:2601.13653* (2026).
  - [72] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110* (2025).
  - [73] Hao Xue and Flora D Salim. 2023. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Transactions on Knowledge and Data Engineering* (2023).
  - [74] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
  - [75] Jinning Yang and Wen Shi. 2025. DiagECG: An LLM-Driven Framework for Diagnostic Reasoning via Discretized ECG Tokenization. *arXiv preprint arXiv:2508.15338* (2025).
  - [76] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
  - [77] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. 2025. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618* (2025).
  - [78] Xuchao Zhang, Yifeng Gao, Jessica Lin, and Chang-Tien Lu. 2020. Tapnet: Multivariate time series classification with attentional prototypical network. In *Proceedings of the AAAI conference on artificial intelligence*.
  - [79] Haokun Zhao, Xiang Zhang, Jiaqi Wei, Yiwei Xu, Yuting He, Siqi Sun, and Chenyu You. 2025. Timeseriescientist: A general-purpose ai agent for time series analysis. *arXiv preprint arXiv:2510.01538* (2025).
  - [80] Ziming Zhao, ChengAo Shen, Hanghang Tong, Dongjin Song, Zhigang Deng, Qingsong Wen, and Jingchao Ni. 2025. From Images to Signals: Are Large Vision Models Useful for Time Series Analysis? *arXiv preprint arXiv:2505.24030* (2025).
  - [81] Jiahui Zhou, Dan Li, Boxin Li, Xiao Zhang, Erli Meng, Lin Li, Zhuomin Chen, Jian Lou, and See-Kiong Ng. 2026. Time Series Reasoning via Process-Verifiable Thinking Data Synthesis and Scheduling for Tailored LLM Reasoning. In *Forty-third International Conference on Machine Learning*.
  - [82] Jiahui Zhou, Dan Li, Lin Li, Zhuomin Chen, Shunyu Wu, Haozheng Ye, Jian Lou, and Costas J Spanos. 2025. Enhancing LLM Reasoning for Time Series Classification by Tailored Thinking and Fused Decision. *arXiv preprint arXiv:2506.00807* (2025).
  - [83] Shu Zhou, Yunyang Xuan, Yuxuan Ao, Xin Wang, Tao Fan, and Hao Wang. 2025. MERIT: Multi-Agent Collaboration for Unsupervised Time Series Representation Learning. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025*.
  - [84] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.
  - [85] Jiaxin Zhuang, Leon Yan, Zhenwei Zhang, Ruiqi Wang, Jiawei Zhang, and Yuntao Gu. 2024. See it, think it, sorted: Large multimodal models are few-shot time series anomaly analyzers. *arXiv preprint arXiv:2411.02465* (2024).
  - [86] Yufan Zhuang, Chandan Singh, Liyuan Liu, Yelong Shen, Dinghui Zhang, Jingbo Shang, Jianfeng Gao, and Weizhu Chen. 2026. Test-time Recursive Thinking: Self-Improvement without External Feedback. *arXiv preprint arXiv:2602.03094* (2026).

## A Appendix Overview

This appendix is organized as follows: We first describe additional implementation details and prompting settings in Appendix B. We then provide supplementary experiments in Appendix C, including extended ablations and more case studies. Finally, Appendix D outlines several promising directions for future research.

## B Additional Implementation Details

**Dataset Selection.** Rather than pursuing exhaustive coverage of the entire UCR/UEA archive, we evaluate MARSTSC on 12 representative datasets spanning diverse domains and temporal characteristics. For more detailed comparisons and ablations, we further adopt a fixed diagnostic subset, which is used consistently across further analysis to ensure comparability and reduce experiment-specific selection bias.

**Hyperparameter Details.** During training, we set the discarding threshold  $\tau_{\text{drop}}$  to 10% of the total training steps, scaling it proportionally with the size of the train set. During testing, by contrast, we heuristically set the promotion threshold  $\tau_{\text{promote}}$  to 5 and the removal threshold  $\tau_{\text{remove}}$  to -2, so as to simulate a real-world deployment scenario where test samples arrive sequentially in a streaming manner.

**Visualization Details.** Line plot is adopted as visual representation. For multivariate datasets, each variate is rendered as an individual subplot with channel index. The original numerical scale is preserved.

**VLMs Details.** We evaluated MARSTSC using 5 representative base model from 4 model families: Gemini [23], GPT [52, 53], Kimi [45], and Qwen [56]. This selection spans diverse parameter scales and is sourced from different corporate developers. Detailed information about these VLMs is summarized in Table 5.

For the main experiments, Gemini serves as the primary base model, while Appendix C.7 reports the results obtained when utilizing the other models. For all experiments, the temperature is fixed at 0.2.

**Prompt Details.** In this section, we provide our prompts for agents in MARSTSC.

### Generator (first-pass generation)

#### ### Task Description

You are an expert in time-series classification.  
Your task is to classify a time-series image into the correct category.

#### ### Instruction

- Analyze the image features and make a judgment based on the provided knowledge bank rules and other provided context.
- You can use the inter-class features to eliminate some categories, and then use intra-class features for pattern matching.
- When there are contradictions among evidence, follow the distinguishing insights. Recheck your thinking

trajectory before output.

#### ### Dataset Information

{ dataset information including dataset name, description, number of classes, sequence length, etc. }

#### ### Query Sample Information

{ statistical information of query sample, including min and max value, mean value, variance, etc. }

#### ### Knowledge Bank

{ knowledge bank }

#### ### Output Format

Output your response strictly in JSON format with the following keys:

- reasoning: your thinking trajectory.
- bullet\_ids: list of the knowledge bank bullet points used in your reasoning.
- final\_answer: your concise final predicted label.

#### ### Query Sample:

Below is the Query Image to be classified:

{ visualized time series }

### Generator (second-pass generation)

#### ### Task Description

You are an expert in time-series classification.  
You are given a reasoning trace generated by another predictor, which should be treated as suspicious evidence, not ground truth.  
Your task is to re-check the classification carefully using the playbook and the query image.

#### ### Instruction

- Act like a second-pass reviewer: explicitly look for errors, overconfidence, and missing evidence in the prior reasoning before deciding.
- If the first-pass reasoning is weak, revise it. If it is strong, you may keep the same answer, but only after independent verification.

#### ### Dataset Information

{ dataset information }

#### ### Query Sample Information

{ statistical information }

#### ### Knowledge Bank

{ knowledge bank }

#### ### First-pass Generator Output

Please re-evaluate the query image provided at the end

**Table 4: Details of datasets**

| Dataset             | Train size | Test size | Classes | Length | Variables | Type      |
|---------------------|------------|-----------|---------|--------|-----------|-----------|
| ArrowHead           | 36         | 175       | 3       | 251    | 1         | IMAGE     |
| DodgerLoopGame      | 20         | 138       | 2       | 288    | 1         | SENSOR    |
| ECG200              | 100        | 100       | 2       | 96     | 1         | ECG       |
| GesturePebbleZ1     | 132        | 172       | 6       | N/A    | 1         | HAR       |
| Lightning7          | 70         | 73        | 7       | 319    | 1         | SENSOR    |
| UMD                 | 36         | 144       | 3       | 150    | 1         | SIMULATED |
| Worms               | 181        | 77        | 5       | 900    | 1         | MOTION    |
| DiatomSizeReduction | 16         | 306       | 4       | 345    | 1         | IMAGE     |
| DistalPhalanxTW     | 400        | 139       | 6       | 80     | 1         | IMAGE     |
| Fish                | 175        | 175       | 7       | 463    | 1         | IMAGE     |
| RacketSports        | 151        | 152       | 4       | 30     | 6         | HAR       |
| SelfRegulationSCP1  | 268        | 293       | 2       | 896    | 4         | EEG       |

**Table 5: Details of adopted base VLMs**

| VLM               | Type        | Parameters                 | Developer   | Release Date |
|-------------------|-------------|----------------------------|-------------|--------------|
| GPT-5.4 mini      | Proprietary | N/A                        | OpenAI      | 2026-03      |
| GPT-5             | Proprietary | N/A                        | OpenAI      | 2025-08      |
| Gemini-3.1 pro    | Proprietary | N/A                        | Google      | 2026-02      |
| Kimi-K2.5         | Open-source | 1000B total / 32B activate | Moonshot AI | 2026-01      |
| Qwen3.5-397B-A17B | Open-source | 397B total / 17B activate  | Alibaba     | 2026-02      |

and classify it.

Use the prior reasoning only as a doubtful clue, and correct it whenever necessary.

{ first-pass generator output }

### ### Output Format

Output your response strictly in JSON format with the following keys:

- reasoning: your thinking trajectory.
- bullet\_ids: list of the knowledge bank bullet points used in your reasoning.
- final\_answer: your concise final predicted label.

### ### Query Sample:

Below is the Query Image to be classified:

{ visualized time series }

Reflector (reflect on correct prediction)

### ### Task Description

You are an expert in time series classification and pattern recognition.

Your task is to analyze a correct classification conducted by the previous predictor and summarize the successful experience.

### ### Instruction

- Analyze the model's reasoning to identify the successful strategies and key insights.
- Evaluate the specific knowledge bank bullet points used

by the model. Were they genuinely useful, neutral, or actually useless but the model succeeded anyway?

### ### Dataset Information

{ dataset information }

### ### Query Sample Information

{ statistical information of query sample }

### ### Knowledge Bank

{ knowledge bank }

### ### Predictor Execution Details

Please analyze this execution and provide your reflection based on the system instructions.

{ truth label of the query sample and output of generator }

### ### Output Format

Output your response strictly in JSON format with the following keys:

- reasoning: your analysis of why the predictor succeeded
- key\_insight: the core successful strategy or experience that should be remembered.
- advice: the advice on how to improve the knowledge bank (empty if not available).
- bullet\_tags: your analysis about the bullet points usage of the predictor. Output in the form of a List[Dict], each dictionary should contain the following keys: (1) id, the index of the bullet point; (2) tag, indicating the role this bullet played in the reasoning process, which must be one of 'useful', 'useless', or 'neutral'; (3) reason, a brief

explanation of your evaluation.

### ### Query Sample:

Below is the Query Image to be classified by the predictor:  
{ visualized time series }

### Reflector (reflect on incorrect prediction)

### ### Task Description

You are an expert in time series classification and pattern recognition.

Your task is to diagnose how the provided erroneous reasoning conducted by a predictor went wrong by analyzing the difference between the predicted label and the ground truth, then distill failure insight.

### ### Instruction

- Carefully analyze the predictor's reasoning trace to identify where it went wrong.
- Identify specific conceptual errors, pattern recognition mistakes, or misapplied strategies.
- Provide actionable insights that could help the model avoid this mistake in the future.
- Focus on the root cause, not just surface-level errors. Be specific about what the model should do to avoid such mistake again.
- You will also receive several sample images from predicted label and the truth label. Identify the differences between these two categories and consider what factors led to incorrect classification in the previous reasoning.

### ### Dataset Information

{ dataset information }

### ### Query Sample Information

{ statistical information of query sample }

### ### Knowledge Bank

{ knowledge bank }

### ### Predictor Execution Details

Please analyze this execution and provide your reflection based on the system instructions.

{ truth label of the query sample and output of generator }

### ### Output Format

Output your response strictly in JSON format with the following keys:

- error\_identification: what specifically went wrong in the original reasoning?
- root\_cause\_analysis: e.g., why did this error occur? What concept was misunderstood? What features are missing or overemphasized in the knowledge bank? What is the confusion point on the decision boundary?

- correct\_reasoning: the correct evidence-based reasoning path

- key\_insight: e.g., what strategy or principle should be remembered to avoid this error?

- bullet\_tags: your analysis of the usage of the bullet points in the predictor. Output in the form of a List[Dict], each dictionary should contain the following keys: (1) id, the index of the bullet point; (2) tag, indicating the role this bullet played in the reasoning process, which must be one of 'useful', 'useless', or 'neutral'; (3) reason, a brief explanation of your evaluation.

### ### Query Sample:

Below is the Query Image to be classified by the predictor:  
{ visualized time series }

### ### Sample in Predicted Label:

{ visualized time series }

### ### Sample in Truth Label:

{ visualized time series }

### Modifier

### ### Task Description

You are a master curator of a time series classification knowledge bank.

The knowledge bank you created will be used to help answer similar classification questions.

Your job is to identify what new insights should be added to an existing knowledge bank, what existing rules should be modified, or what to delete based on a reflection from a previous attempt.

### ### Instruction

- Review the existing knowledge bank and the reflection from the previous reasoning.
- Identify ONLY the NEW insights, strategies, or mistakes that are MISSING from the current knowledge bank, or identify rules that need modification/deletion.
- Be concise and specific; each bullet point must be actionable.
- Return an empty list in the operations field if there is no need to modify the current knowledge bank.

### ### Dataset Information

{ dataset information }

### ### Query Sample Information

{ statistical information of query sample }

### ### Knowledge Bank

{ knowledge bank }

### ### Predictor Execution Details

{ truth label of the query sample and output of generator }

### ### Previous Reflection

{ output of reflector }

### ### Output Format

Output your response strictly in JSON format with the following keys:

- reasoning: your thinking trajectory.
- operations: operations you plan to perform on the knowledge bank.

Available Operations:

1. ADD: Create new bullet points with fresh IDs.
  - section: the section to add the new bullet to.
  - content: the new content of the bullet.
2. MODIFY: Update an existing bullet point.
  - target\_id: the exact ID of the bullet to modify.
  - content: the fully updated content of the bullet.
3. DELETE: Remove an existing bullet point.
  - target\_id: index of the bullet point you want to remove.

### ### Query Sample:

Below is the Query Image to be classified by the predictor:  
{ visualized time series }

## C Detailed Results & Extended Experiments

### C.1 Detailed Results of Increasing-Shot Comparison

Figure 3 in the main paper visualizes the overall trend of the comparison between MARSTSC under  $k$ -shot setting and baselines under increasing  $k$ -shot setting. Table 6 reports the detailed numerical results to better clarify the performance of MARSTSC.

### C.2 Comparison with Extended Baselines

We add the following four new baselines in comparison: **DTW** [8]: a classical few-shot machine learning method; **COSCO** [6] and its earlier baseline **TapNet** [78]: two deep learning-based methods *designed with few-shot-tailored adaptation*, and **TimeOmni-VL** [26]: a time-series VLM, which serves as a representative baseline with *cross-modal alignment capability*. For DTW and TapNet, we use the implementation from AEON [41]. For COSCO, we train it on few-shot train set with default settings as detailed in the paper, i.e.,  $lr = 0.01$  and 100 epochs. For TimeOmni-VL, we use it without fine-tuning, provide the same prompt and knowledge bank as MARSTSC to supply task background, while keeping all generation hyperparameters at default values.

Table 7 shows that MARSTSC consistently outperforms both few-shot-tailored and cross-modal-aligned baselines. This result supports that the performance gain mainly comes from the proposed agentic reasoning framework. In particular, the comparison with COSCO shows the advantage of MARSTSC over dedicated

few-shot methods. The comparison with TimeOmni-VL further indicates that directly applying a TS-aligned VLM remains insufficient without structured reflective refinement and knowledge evolution.

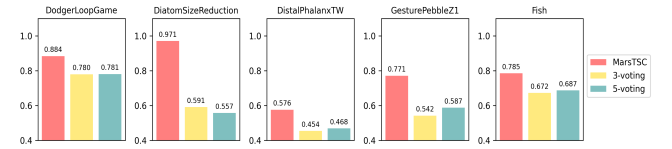
**Table 7: Accuracy Comparison with extended baselines**

| Dataset             | MARSTSC      | DTW   | COSCO | TapNet | TimeOmni-VL |
|---------------------|--------------|-------|-------|--------|-------------|
| DodgerLoopGame      | <b>0.884</b> | 0.662 | 0.567 | 0.642  | 0.541       |
| DiatomSizeReduction | <b>0.971</b> | 0.906 | 0.952 | 0.590  | 0.265       |
| DistalPhalanxTW     | <b>0.576</b> | 0.542 | 0.528 | 0.515  | 0.212       |
| Fish                | <b>0.771</b> | 0.564 | 0.661 | 0.496  | 0.145       |
| GesturePebbleZ1     | <b>0.785</b> | 0.648 | 0.732 | 0.519  | 0.165       |

### C.3 Analysis on Test-Time Computation

To isolate the effect of test-time computation, we conduct a new experiment to compare our method with a test-time scaling variant based on  $m$ -voting self-consistency. The rationale for this new comparison is that it provides a scenario of *spending comparable inference budget without agentic reasoning*. Specifically, for a dataset with  $c$  classes, we perform  $m \times c$  inference calls and take the majority vote as the final prediction. The voting variant uses the same prompt as Generator agent and warm-up knowledge bank.

As shown in Figure 6, increased computation budget can improve performance, but they still underperform the full MARSTSC pipeline. This suggests that larger test-time computation alone cannot explain the observed gains. The key advantage of MARSTSC lies in how the computation is structured: instead of simply aggregating multiple predictions, MARSTSC uses insight distillation and cautious deferred updates to convert reasoning outcomes into reusable evolving knowledge.



**Figure 6: Comparison with test-time scaling method**

### C.4 Analysis of Few-shot Train Samples

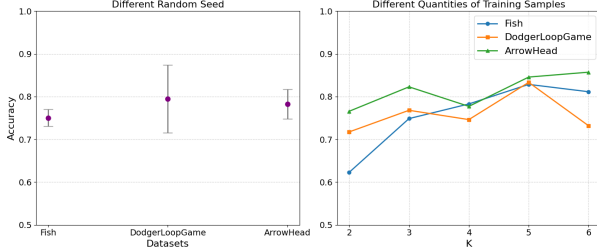
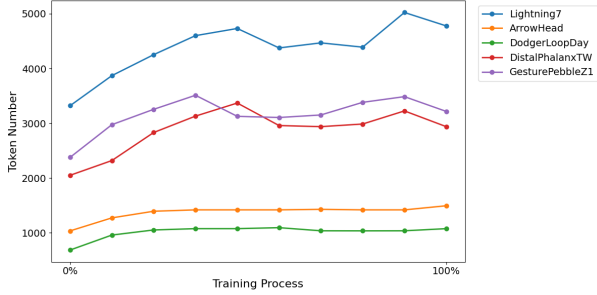
To further evaluate the robustness of MARSTSC under few-shot sampling randomness and varying levels of data scarcity, we conduct extended experiments on three datasets with different  $k$ -shot settings ( $k \in \{2, 3, 4, 5, 6\}$ ) and random seeds. Figure 7 shows that the error bars remain relatively small, indicating stable performance across different few-shot selections. In addition, the mean accuracy generally improves as the number of shots increases, demonstrating that the dynamic knowledge bank can progressively incorporate new discriminative patterns in an effective and stable manner. While a minor fluctuation appears on DodgerLoopGame at the 6-shot setting, likely caused by anomalous samples, the overall trend confirms the strong reliability and generalization ability of MARSTSC across diverse few-shot conditions.

### C.5 Analysis of Knowledge Bank Size

To further analyze the behavior of the evolving knowledge bank, we monitor its token count during training across different time-series datasets. Figure 8 shows that the token count increases only in the

**Table 6: Detailed results of comparison with baselines under increasing-shot**

| Dataset         | Ours         | TimesNet |       |       |       |           | PatchTST |       |       |       |           | GPT4TS   |       |       |       |           | MOMENT   |       |       |       |              |
|-----------------|--------------|----------|-------|-------|-------|-----------|----------|-------|-------|-------|-----------|----------|-------|-------|-------|-----------|----------|-------|-------|-------|--------------|
|                 | <i>k</i>     | <i>k</i> | 2k    | 3k    | 4k    | Full-shot | <i>k</i> | 2k    | 3k    | 4k    | Full-shot | <i>k</i> | 2k    | 3k    | 4k    | Full-shot | <i>k</i> | 2k    | 3k    | 4k    | Full-shot    |
| DodgerLoopGame  | <b>0.884</b> | 0.681    | 0.761 | 0.761 | 0.833 | 0.833     | 0.522    | 0.572 | 0.739 | 0.638 | 0.638     | 0.529    | 0.565 | 0.695 | 0.699 | 0.699     | 0.710    | 0.717 | 0.818 | 0.804 | 0.804        |
| DistalPhalanxTW | 0.576        | 0.467    | 0.518 | 0.573 | 0.612 | 0.705     | 0.144    | 0.468 | 0.417 | 0.511 | 0.626     | 0.612    | 0.612 | 0.604 | 0.604 | 0.705     | 0.511    | 0.554 | 0.547 | 0.576 | <b>0.727</b> |
| Fish            | 0.771        | 0.506    | 0.571 | 0.691 | 0.726 | 0.829     | 0.183    | 0.331 | 0.320 | 0.480 | 0.491     | 0.286    | 0.291 | 0.371 | 0.377 | 0.440     | 0.709    | 0.726 | 0.857 | 0.869 | <b>0.914</b> |
| GesturePebbleZ1 | <b>0.785</b> | 0.302    | 0.343 | 0.302 | 0.349 | 0.390     | 0.204    | 0.209 | 0.180 | 0.308 | 0.308     | 0.168    | 0.157 | 0.192 | 0.221 | 0.250     | 0.198    | 0.302 | 0.343 | 0.308 | 0.401        |

**Figure 7: Left: Comparison of mean accuracy and standard deviation with different random seeds; Right: Accuracy of three datasets across different *k*-shot settings.****Figure 8: Token count of the knowledge bank over the course of training.**

early stage and then gradually stabilizes, rather than expanding without bound. This suggests that the proposed bullet-level update mechanism is able to selectively preserve informative knowledge while filtering out redundant or low-value content. Such a design mitigates the risk of context degradation or collapse caused by repeatedly rewriting the entire knowledge bank, thereby allowing agents to operate with a compact, discriminative, and computationally efficient context during iterative reasoning.

### C.6 Ablation on Key Stages

To understand the contribution of each stage, we compare the full pipeline with two reduced variants: *w/o test update* disables test-time update, and *w/o train refine & test update*, disables both train-time refinement and test-time update, which performs testing solely based on the initial statistical features and visual descriptions. Notably, *w/o train refine & test update* can be regarded as a setting that mainly reflects the classification ability of the VLM itself, since no agentic operation is involved.

As shown in Table 8, *w/o test update* consistently outperforms *w/o both* on all datasets, indicating that statistical features and visual descriptions alone are insufficient for effective classification, highlighting the importance of the agent reasoning, whose role is

**Table 8: Ablation on key stages**

| Dataset             | Full pipeline | w/o test update | w/o train refine & test update |
|---------------------|---------------|-----------------|--------------------------------|
| ArrowHead           | <b>0.766</b>  | 0.691           | 0.543                          |
| DodgerLoopGame      | <b>0.884</b>  | 0.812           | 0.797                          |
| ECG200              | <b>0.790</b>  | 0.770           | 0.640                          |
| GesturePebbleZ1     | <b>0.785</b>  | 0.756           | 0.744                          |
| Lightning7          | <b>0.822</b>  | 0.795           | 0.740                          |
| UMD                 | <b>0.993</b>  | 0.993           | 0.924                          |
| Worms               | <b>0.481</b>  | 0.464           | 0.442                          |
| DiatomSizeReduction | <b>0.971</b>  | 0.971           | 0.771                          |
| DistalPhalanxTW     | <b>0.576</b>  | 0.561           | 0.510                          |
| Fish                | <b>0.771</b>  | 0.726           | 0.600                          |
| RacketSports        | <b>0.526</b>  | 0.526           | 0.454                          |
| SelfRegulationSCP1  | 0.697         | <b>0.717</b>    | 0.519                          |

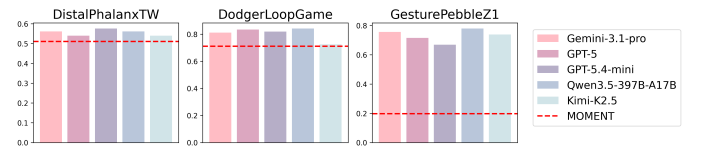
not to directly make the final prediction but to accumulate successful experience, extract corrective feedback from failures, and refine the inference process accordingly.

Moreover, in most cases, the full pipeline further outperforms *w/o test update*, suggesting that the proposed test-time update mechanism can uncover discriminative patterns not fully covered by the few-shot samples, thereby alleviating the limited representativeness and generalization of few-shot supervision.

### C.7 Analysis of Base VLM Sensitivity

To further investigate the robustness of our method across different foundation models, we additionally evaluate MARSTSC with four other mainstream VLMs, including GPT-5 [52], GPT-5.4 mini [53], Qwen-3.5-397B-A17B [56], and Kimi-K2.5 [45].

As illustrated in Figure 9, the overall performance of our method varies slightly with the capability of the base VLM, which is expected given the different reasoning and multimodal understanding abilities of these models. Nevertheless, in most cases, our method still outperforms most baselines shown in Table 1. This suggests that our method can be effectively transferred across different VLM backbones while retaining strong performance.

**Figure 9: Classification accuracy with different base VLMs**

## C.8 More Case Studies

Figure 10 presents a successful example of test-time self-recheck via two-pass generation, where the model revisits its previous reasoning, corrects the initial mistake, and ultimately arrives at the correct answer. This case suggests that the test time design can effectively reduce the risk of error-induced knowledge contamination while further expanding the coverage and adaptability of the knowledge memory.

We further present two failure cases to clarify the effective boundary of the proposed two-pass design. In the first case, as shown in Figure 11, both passes produce the same incorrect prediction, indicating that the second pass does not always constitute a genuinely independent re-evaluation. Instead, it may remain anchored to the initial hypothesis and further reinforce a self-consistent but incorrect reasoning. Figure 12 provides the second case, in which the first pass gives the correct answer, whereas the second pass revises it to an incorrect one. This suggests that self-recheck may occasionally introduce over-correction, where additional reasoning shifts attention toward misleading or non-discriminative cues rather than improving the original judgment.

Taken together, these cases show that two-pass generation should be understood as a risk-reduction mechanism rather than a guarantee of correction. While it can often correct initial mistakes, its effectiveness depends on whether the second pass can both escape first-pass anchoring and rely on sufficient discriminative knowledge. This observation also motivates our conservative deferred knowledge update strategy during test time, which avoids directly treating second-pass outputs as fully trustworthy knowledge.

## D Future Directions

Several promising directions warrant further investigation. First, a unified time-series reasoning framework could extend the proposed MARS-TSC beyond classification to forecasting, anomaly detection, imputation, and temporal question answering. Such a framework would require task-aware knowledge structures and reasoning policies that capture both shared temporal characteristics and task-specific decision objectives. Second, richer visual representations, such as spectrograms or recurrence plots, deserve systematic exploration. These representations may help VLM agents better capture frequency-domain patterns that are difficult to identify from line plots alone. Third, integrating more capable agents with tool-use and long-term memory represents another important direction. External tools can support evidence verification, while persistent memory can retain reusable knowledge across datasets and tasks, improving the reliability and transferability of MARS-TSC.



**Figure 10: A case about self-recheck in test time. By using prior reasoning as suspicious reference, generator successfully identified the original mistake and corrected it.**

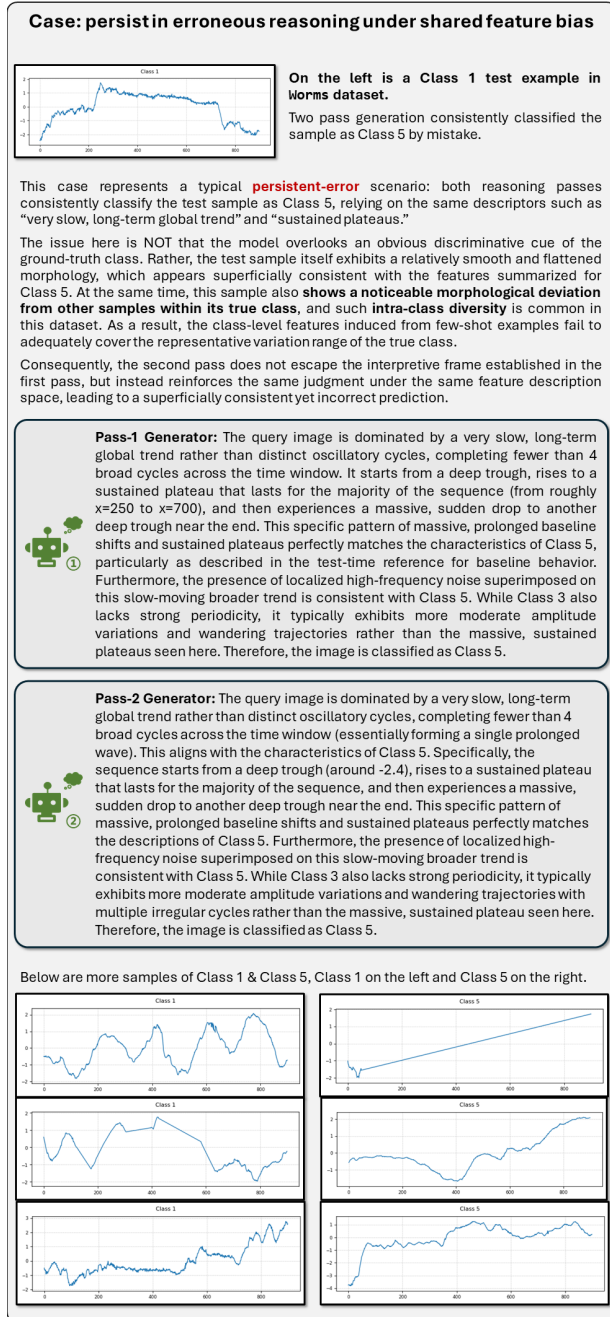


Figure 11: A case about self-recheck in test time. Pass-2 generation failed to correct the answer due to insufficient coverage of pattern diversity in the few-shot examples.



Figure 12: A case about self-recheck in test time. Pass-2 reasoning revises a correct initial prediction due to an ambiguous description in the knowledge bank.